

Nasywaa Kamila_Calcite

Tugas Fase 1 — Pemrograman TLS26

Problem 1: The Last Astronaut

Pseudocode

```
BEGIN
  INPUT N, K
  buat list astronot berisi 1, 2, ..., N
  current <- 0
  eliminationOrder <- [ ]

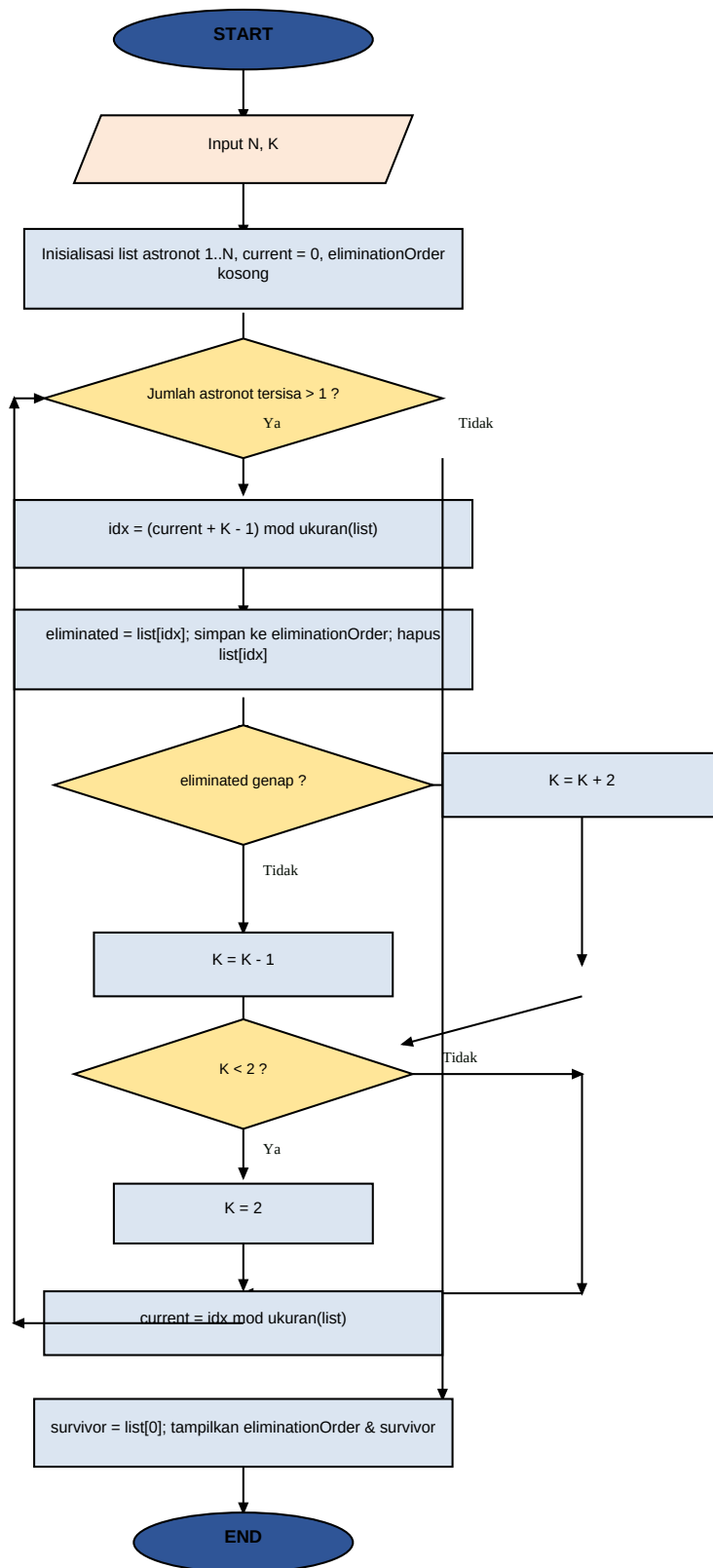
  WHILE ukuran(astronot) > 1 DO
    idx <- (current + K - 1) MOD ukuran(astronot)
    eliminated <- astronot[idx]
    TAMBAHKAN eliminated ke eliminationOrder
    HAPUS astronot[idx] dari list

    IF eliminated MOD 2 = 0 THEN
      K <- K + 2
    ELSE
      K <- K - 1
    END IF
    IF K < 2 THEN
      K <- 2
    END IF

    current <- idx MOD ukuran(astronot)
  END WHILE

  survivor <- astronot[0]
  OUTPUT eliminationOrder
  OUTPUT survivor
END
```

Flowchart



Penjelasan Logika

Persoalan ini merupakan variasi dari Josephus Problem klasik, dengan perbedaan utama bahwa nilai K tidak tetap, melainkan berubah setiap kali terjadi eliminasi. Karena perubahan K bergantung pada paritas (genap/ganjil) nomor astronot yang baru saja dieliminasi, penyelesaian tidak dapat menggunakan rumus matematis Josephus standar dan harus diselesaikan dengan simulasi langkah demi langkah. Astronot direpresentasikan sebagai sebuah struktur melingkar (dapat berupa array dinamis atau circular linked list) yang elemennya dihapus satu per satu hingga tersisa satu astronot.

Pada setiap iterasi, posisi astronot yang akan dieliminasi dihitung dengan bergerak sejauh K langkah dari posisi penghitungan saat ini, menggunakan operasi modulo terhadap jumlah astronot yang masih tersisa (bukan N awal), karena ukuran lingkaran terus mengecil setiap eliminasi. Setelah sebuah astronot dieliminasi dan dicatat ke dalam daftar urutan eliminasi, nilai K diperbarui sesuai aturan paritas, dengan batas bawah K tidak boleh kurang dari 2. Posisi awal hitungan berikutnya (current) juga diperbarui agar menunjuk ke astronot tepat setelah yang baru dieliminasi. Proses ini diulang dalam sebuah perulangan (loop) hingga hanya tersisa satu astronot, yang kemudian ditetapkan sebagai pemenang (survivor).

Problem 2: Alien-In-The-Middle

Pseudocode

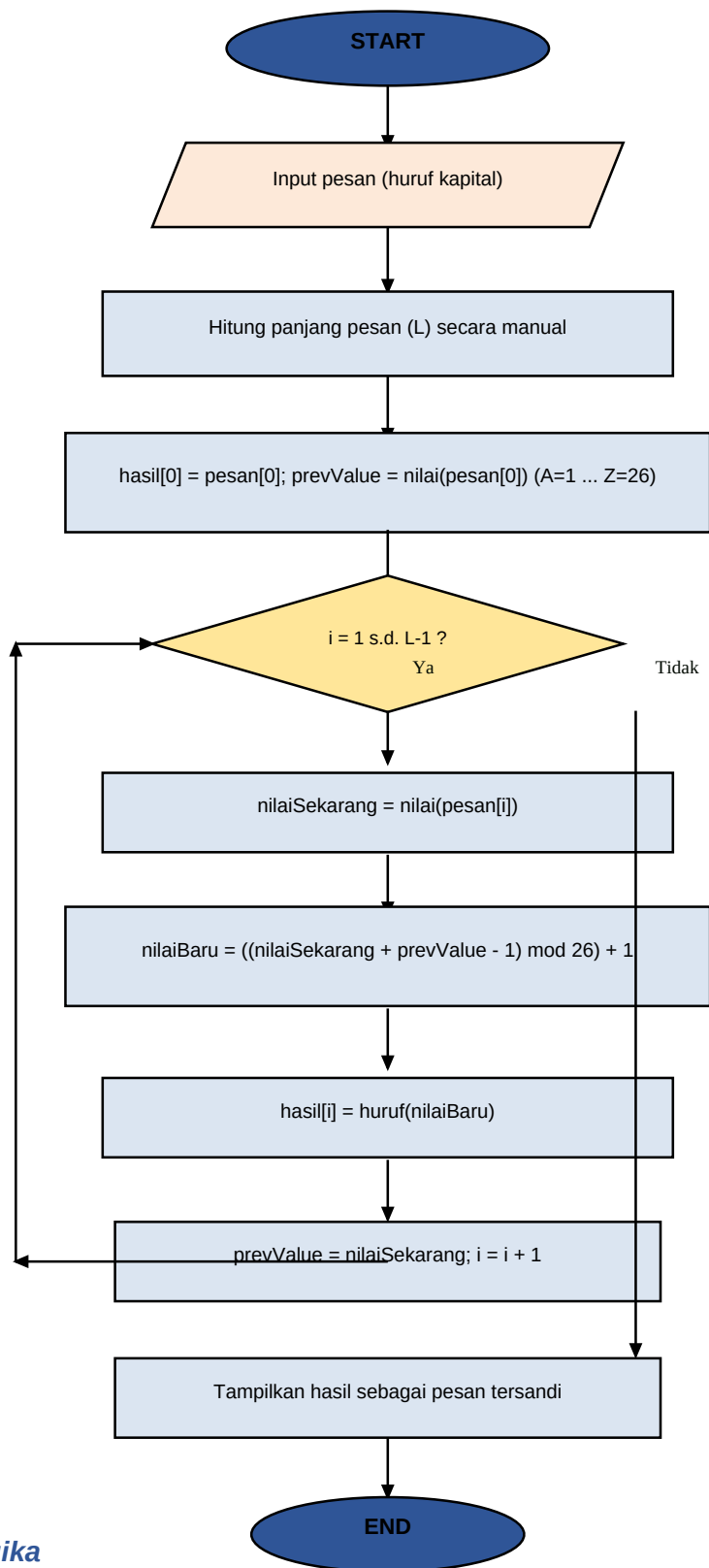
```
BEGIN
  INPUT pesan (string huruf kapital)
  L <- panjang(pesan)           // dihitung manual, tanpa fungsi library

  hasil[0] <- pesan[0]
  prevValue <- ANGKA(pesan[0])  // A=1, B=2, ..., Z=26

  FOR i <- 1 TO L-1 DO
    nilaiSekarang <- ANGKA(pesan[i])
    nilaiBaru <- ((nilaiSekarang + prevValue - 1) MOD 26) + 1
    hasil[i] <- HURUF(nilaiBaru) // nilaiBaru -> 'A'..'Z'
    prevValue <- nilaiSekarang   // nilai ASLI, bukan nilai hasil sandi
  END FOR

  OUTPUT hasil sebagai pesan tersandi
END
```

Flowchart



Penjelasan Logika

Sandi pada persoalan ini merupakan bentuk pergeseran huruf (mirip Caesar cipher), tetapi nilai pergeserannya tidak konstan, melainkan bergantung pada nilai huruf asli yang berada tepat sebelumnya dalam pesan. Setiap huruf dipetakan ke nilai numerik posisinya di alfabet (A=1 hingga Z=26). Huruf pertama tidak mengalami perubahan karena tidak memiliki huruf sebelumnya sebagai acuan pergeseran, sehingga huruf tersebut langsung disalin ke pesan hasil dan nilainya disimpan sebagai acuan pergeseran huruf berikutnya.

Untuk huruf kedua dan seterusnya, nilai huruf saat ini dijumlahkan dengan nilai huruf asli sebelumnya, kemudian hasilnya disesuaikan agar tetap berada dalam rentang 1-26 menggunakan rumus $((\text{nilaiSekarang} + \text{prevValue} - 1) \bmod 26) + 1$, yang secara otomatis menangani kondisi saat hasil penjumlahan melewati Z (wrap-around kembali ke A) tanpa memerlukan percabangan IF terpisah. Hal yang harus dijaga secara cermat adalah bahwa nilai "sebelumnya" yang digunakan untuk menggeser huruf berikutnya selalu diambil dari huruf ASLI pesan, bukan dari huruf yang sudah tersandikan, sesuai dengan pola pada tabel contoh ALIENS -> AMUNSG. Proses ini diulang untuk seluruh huruf dalam pesan hingga terbentuk pesan akhir yang telah tersandi.